

PYTHON • OBJECT-ORIENTED PROGRAMMING

Object-Oriented Programming, from blueprint to running system.

A complete, hands-on tutorial that takes you from your first `class` keyword to designing composable, Pythonic object systems — with worked examples, diagrams, exercises, and graded quizzes at every step.

LIVE DEMO • CLASS → OBJECT

```
class Robot:
    name    → ?
    battery → 100
    def greet(self): ...
```

Click the button above a few times — each click builds a real object from the same blueprint, each with its own data. That's the whole idea of OOP in one gesture.

ESTIMATED TIME

2.5 – 3.5 hours

PREREQUISITES

Python basics: variables,
functions, lists

FORMAT

4 modules · 16 quiz
questions

OUTCOME

Design real class hierarchies
confidently

Learning objectives

- Explain and apply classes, objects, attributes, and methods correctly
- Use the four OOP pillars — encapsulation, inheritance, polymorphism, abstraction — in real code
- Distinguish instance vs. class-level state and know when to use each
- Read and write dunder methods, properties, and composition-based designs
- Recognize and avoid the most common OOP mistakes Python beginners make

Reward system



Foundations Builder



Pillar Architect



Practical Engineer



OOP Master

MOD 01

Foundations

CLASSES · OBJECTS · ATTRIBUTES · METHODS · `__INIT__` · `SELF`

What a class actually is

A **class** is a blueprint. It doesn't *do* anything by itself — it describes what data an object will hold (attributes) and what it can do (methods). An **object** (or *instance*) is a concrete thing built from that blueprint, living in memory with its own values.

ANALOGY

A cookie cutter is the class. Every cookie you stamp out is an object. All cookies share the same shape (defined by the cutter), but each one is a separate cookie — decorate one with sprinkles and the others stay untouched. Same blueprint, independent instances.

Defining a class

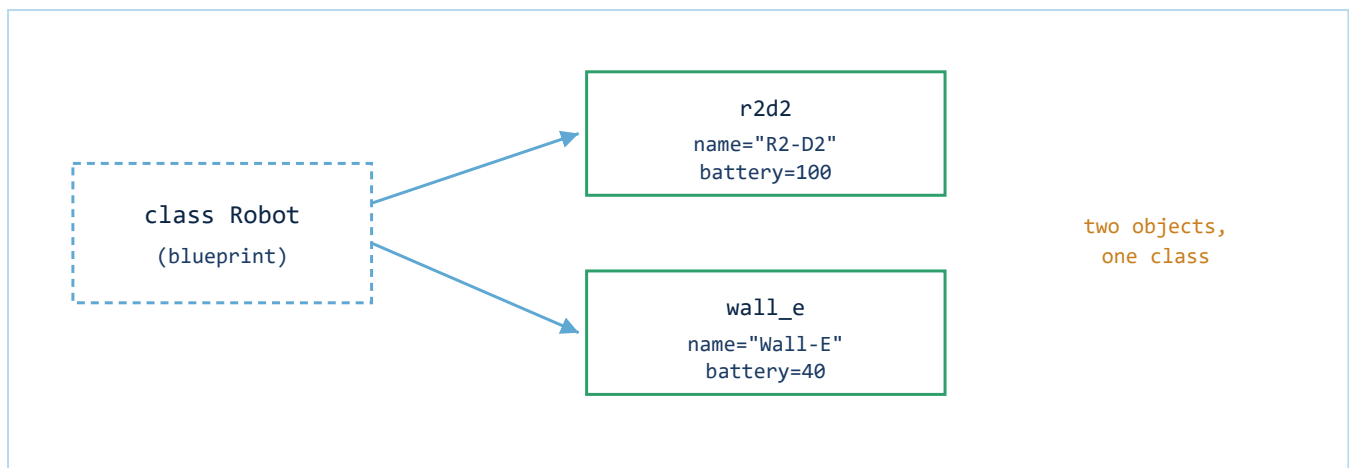
```
class Robot:
    def __init__(self, name, battery=100):
        self.name = name          # instance attribute
        self.battery = battery    # instance attribute

    def greet(self):
        return f"Beep boop, I am {self.name}."

r2d2 = Robot("R2-D2")
wall_e = Robot("Wall-E", battery=40)

print(r2d2.greet())              # Beep boop, I am R2-D2.
print(wall_e.battery)            # 40
```

`__init__` is the **constructor**: Python calls it automatically the moment you write `Robot("R2-D2")`. `self` is simply the object being built — every method receives it as the first parameter so it can read and modify that specific object's data.



Predict the output

Given the `Robot` class above, what does `print(r2d2.battery)` output?

Class attributes vs. instance attributes

	INSTANCE ATTRIBUTE	CLASS ATTRIBUTE
Defined	Inside <code>__init__</code> via <code>self.x = ...</code>	Directly in the class body
Storage	Unique copy per object	Shared by all objects
Typical use	Data that varies per object (name, battery)	Constants or shared counters (species, max_units)
Example	<code>self.name = name</code>	<code>units_built = 0</code>

COMMON MISCONCEPTION

"`self` is a Python keyword." It isn't — it's just a conventional parameter name. You could call it anything, but every Python developer expects `self`, so deviating from it makes code harder to read for no benefit.

EXERCISE

Write a `Book` class with `title`, `author`, and `pages` attributes, plus a method `summary()` that returns a formatted string. Try it below (this is a scratchpad — it isn't executed, just for practice).

KEY TAKEAWAYS

- A class is a blueprint; an object is a specific instance built from it
- `__init__` runs automatically to set up a new object's state

- `self` refers to "this particular object" inside a method
- Instance attributes vary per object; class attributes are shared

MOD 02

The Four Pillars

ENCAPSULATION • INHERITANCE • POLYMORPHISM • ABSTRACTION

Encapsulation: protecting internal state

Encapsulation bundles data and behavior together and controls how that data is accessed. Python doesn't have true "private" variables, but a leading underscore signals "internal use only," and a double underscore triggers name mangling to discourage accidental access.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # name-mangled, harder to touch directly

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

acct = BankAccount(100)
acct.deposit(50)
print(acct.get_balance())    # 150
```

ANALOGY

A vending machine encapsulates its cash box. You don't reach in and grab coins — you interact through the coin slot and buttons (the public methods), while the machine controls its internal state safely.

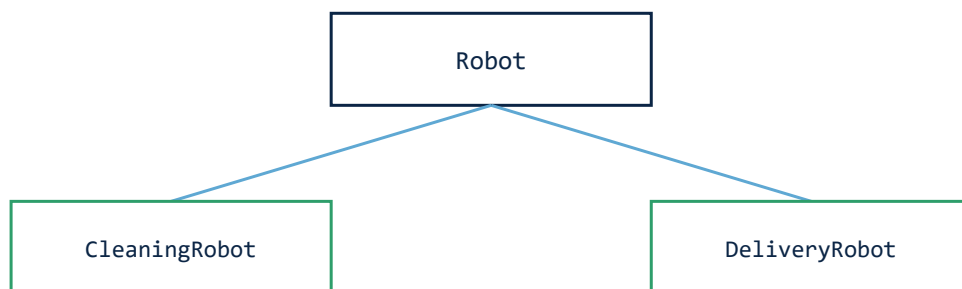
Inheritance: building on existing blueprints

Inheritance lets a new class reuse and extend an existing one. The child class inherits attributes and methods, and can override or add to them.

```
class Robot:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"I am {self.name}."

class CleaningRobot(Robot):          # inherits from Robot
    def clean(self):
        return f"{self.name} is vacuuming."

rosie = CleaningRobot("Rosie")
print(rosie.greet())  # inherited method still works
print(rosie.clean())  # new method from the subclass
```



Polymorphism: same call, different behavior

```
class DeliveryRobot(Robot):
    def greet(self):                # overrides Robot.greet
        return f"Package incoming from {self.name}!"

for bot in [CleaningRobot("Rosie"), DeliveryRobot("Dash")]:
    print(bot.greet())  # each object responds in its own way
```

Polymorphism means you can call `.greet()` on any `Robot`-family object without knowing its exact subclass — each one supplies its own behavior.

Abstraction: hiding the "how"

Abstraction exposes only what's necessary. Python's `abc` module lets you define abstract methods that subclasses *must* implement, enforcing a contract without dictating the internal details.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
```

```
pass # subclasses MUST implement this
```

```
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2
```

PILLAR	SOLVES	PYTHON MECHANISM
Encapsulation	Uncontrolled access to internal state	<code>_ / __</code> naming, properties
Inheritance	Duplicated code across related classes	<code>class Child(Parent):</code>
Polymorphism	Rigid code that checks types everywhere	Method overriding, duck typing
Abstraction	Callers depending on implementation details	<code>abc.ABC</code> , abstract methods

COMMON MISCONCEPTION

"Inheritance means the child class must use everything the parent gives it." Not true — a subclass can override any method to change behavior, and Python's dynamic nature means it can even ignore inherited methods entirely if it defines its own.

Predict the output

Given the code above, what does `DeliveryRobot("Dash").greet()` return?

"I am Dash."

"Package incoming from Dash!"

Error – greet is already defined

EXERCISE

Create an abstract `Employee` class with an abstract method `pay()` , then two subclasses `SalariedEmployee` and `HourlyEmployee` that implement it differently.

Show a model answer

KEY TAKEAWAYS

- Encapsulation controls access; use `_ / __` and methods, not raw exposure
- Inheritance shares code via `class Child(Parent):` and can override methods
- Polymorphism lets different objects respond to the same call in their own way
- Abstraction defines a contract via `abc.ABC` without fixing the implementation

MOD 03

Practical Application

DUNDER METHODS • PROPERTIES • CLASS METHODS & STATIC METHODS • COMPOSITION

Dunder methods: making your objects feel native

"Dunder" (double underscore) methods hook your objects into Python's built-in syntax — printing, comparison, length, arithmetic — so they behave like first-class citizens rather than bare data containers.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

v1, v2 = Vector(1, 2), Vector(3, 4)
print(v1 + v2)          # Vector(4, 6) - uses __add__
print(v1 == Vector(1, 2)) # True - uses __eq__
```

ANALOGY

Dunder methods are like standardized power outlets. Any appliance (object) that has the right plug shape (implements `__add__`, `__len__`, etc.) can be plugged into Python's built-in syntax — `+`, `len()`, `print()` — without special cases.

DUNDER	TRIGGERED BY	TYPICAL USE
<code>__init__</code>	<code>ClassName(...)</code>	Set up initial state
<code>__repr__</code>	<code>print(obj)</code> , console display	Unambiguous debug representation
<code>__eq__</code>	<code>obj1 == obj2</code>	Define value equality
<code>__len__</code>	<code>len(obj)</code>	Give objects a "size"
<code>__add__</code>	<code>obj1 + obj2</code>	Custom arithmetic

Properties: controlled attribute access

```

class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def area(self):
        return 3.14159 * self._radius ** 2

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value <= 0:
            raise ValueError("radius must be positive")
        self._radius = value

c = Circle(4)
print(c.area)      # computed on access, called like an attribute
c.radius = 10      # goes through validation in the setter

```

A `@property` lets you call a method as if it were a plain attribute — giving you validation and computed values without changing how callers use the class.

Class methods vs. static methods vs. instance methods

```

class Robot:
    units_built = 0

    def __init__(self, name):
        self.name = name
        Robot.units_built += 1

    def greet(self):                                # instance method – needs self
        return f"I am {self.name}"

    @classmethod
    def from_default(cls):                          # works with the class, not one object
        return cls("Unit-Default")

    @staticmethod
    def is_valid_name(name):                        # doesn't need self or cls at all
        return len(name) > 0

```


	FIRST PARAM	CAN ACCESS INSTANCE DATA?	TYPICAL USE
Instance method	self	Yes	Normal behavior on one object
Class method	cls	No, only class-level	Alternate constructors, factory methods
Static method	none	No	Utility function related to the class conceptually

Composition: "has-a" instead of "is-a"

Inheritance models "is-a" relationships. When that doesn't fit, prefer **composition** — building an object out of other objects.

```
class Engine:
    def start(self): return "Engine roaring"

class Car:                                # a Car HAS an Engine (not IS an Engine)
    def __init__(self):
        self.engine = Engine()
    def start(self):
        return self.engine.start()
```

COMMON MISCONCEPTION

"Inheritance is always the right way to reuse code." Overusing inheritance creates deep, fragile hierarchies. The classic guideline: favor composition over inheritance when the relationship isn't a clean "is-a" — a Car is not an Engine, but it has one.

Predict the output

Given the `Circle` class above, what happens when you run `c.radius = -5` ?

Silently sets radius to -5

A `ValueError` is raised

An `AttributeError` is raised

EXERCISE

Add `__len__` and `__str__` to a `Playlist` class that wraps a list of songs, so

`len(playlist)` and `print(playlist)` both work naturally.

Show a model answer

KEY TAKEAWAYS

- Dunder methods let your objects work with Python's native syntax (`+` , `len()` , `print()`)
- `@property` gives attribute-like access with validation or computed logic

- Use `@classmethod` for alternate constructors, `@staticmethod` for related utilities
- Prefer composition ("has-a") over inheritance ("is-a") when the relationship isn't a true hierarchy

MOD 04

Mastery & Design

MULTIPLE INHERITANCE & MRO • MIXINS • DATACLASSES • DESIGN PATTERNS • BEST PRACTICES

Multiple inheritance and the MRO

Python allows a class to inherit from more than one parent. When methods collide, Python resolves the call order using the **Method Resolution Order (MRO)** — a predictable, linear path computed with the C3 linearization algorithm.

```
class Flyable:
    def move(self): return "Flying"

class Swimmable:
    def move(self): return "Swimming"

class Duck(Flyable, Swimmable):
    pass

print(Duck().move())          # "Flying" – Flyable is listed first
print(Duck.__mro__)          # shows the exact lookup order
```

ANALOGY

Think of MRO as a chain of command. When `Duck` needs `move()`, Python asks each "superior" in a fixed order — `Duck`, then `Flyable`, then `Swimmable` — and uses the first answer it finds.

Mixins: small, focused, reusable behavior

```
class JSONMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class User(JSONMixin):
    def __init__(self, name, age):
        self.name, self.age = name, age
```

```
print(User("Ada", 30).to_json()) # {"name": "Ada", "age": 30}
```

A mixin is a small class designed purely to be combined with others — it isn't meant to stand alone. It adds one capability (serialization, comparison, logging) without forcing a rigid inheritance tree.

Dataclasses: less boilerplate for data-holding classes

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
print(p)           # Point(x=1.0, y=2.0) – auto-generated __repr__
print(p == Point(1.0, 2.0)) # True – auto-generated __eq__
```

`@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` for classes whose main job is holding data, cutting out repetitive boilerplate.

A first taste of design patterns

PATTERN	PROBLEM IT SOLVES	SHAPE
Factory Method	Object creation logic scattered everywhere	A <code>classmethod</code> or function that returns the right subclass
Strategy	Hard-coded conditional behavior	Swap in different behavior objects at runtime
Observer	Objects need to react to another object's changes	Subject notifies a list of registered listeners
Singleton	Exactly one instance should ever exist	Class controls its own single instantiation

```
# Strategy pattern, minimal example
class Robot:
    def __init__(self, movement_strategy):
        self.movement_strategy = movement_strategy
    def move(self):
        return self.movement_strategy()

wheeled = Robot(lambda: "Rolling on wheels")
legged = Robot(lambda: "Walking on legs")
```

COMMON MISCONCEPTION

"More inheritance levels and design patterns automatically mean better code." In practice, over-engineering with deep hierarchies or unnecessary patterns makes code harder to follow. Python favors simple, explicit designs — reach for a pattern only when it removes real duplication or rigidity.

Predict the output

Using the `Duck(Flyable, Swimmable)` example, what does `Duck().move()` return?

EXERCISE

Write a `LoggingMixin` with a `log(message)` method, then combine it with a `Robot` class using multiple inheritance.

KEY TAKEAWAYS

- The MRO gives Python a predictable, linear order for resolving multiple inheritance
- Mixins add one focused capability and aren't meant to be instantiated alone
- `@dataclass` removes boilerplate for data-centric classes
- Design patterns solve recurring problems — use them when they reduce complexity, not by default

FINAL

Final Challenge & Reference

Final practical challenge

Design a small `Library` system that ties everything together:

- An abstract `Media` base class (abstraction) with an abstract `describe()` method
- `Book` and `DVD` subclasses that override `describe()` differently (polymorphism, inheritance)
- A `Library` class that *has* a list of `Media` items (composition), with a private `__catalog` (encapsulation)
- A `__len__` dunder method so `len(library)` returns the item count
- A `@classmethod` `Library.empty()` that returns a new, empty library

BUILD IT

Hide model answer

```
from abc import ABC, abstractmethod

class Media(ABC):
    def __init__(self, title):
        self.title = title
    @abstractmethod
    def describe(self): pass

class Book(Media):
    def describe(self):
        return f"Book: {self.title}"

class DVD(Media):
    def describe(self):
        return f"DVD: {self.title}"

class Library:
    def __init__(self):
        self.__catalog = []

    @classmethod
    def empty(cls):
        return cls()

    def add(self, item):
        self.__catalog.append(item)

    def __len__(self):
        return len(self.__catalog)

lib = Library.empty()
lib.add(Book("Dune"))
lib.add(DVD("Arrival"))
print(len(lib))    # 2
```

Cheat sheet

Define & construct

```
class Name: def __init__(self, ...): self.x = x
```

Inherit

```
class Child(Parent): ... · call parent with super().__init__(...)
```

Encapsulate

`self._internal` (convention) · `self.__private` (name-mangled)

Polymorphism

Override a method in a subclass; call it the same way on any subclass instance

Abstraction

`from abc import ABC, abstractmethod + @abstractmethod`

Dunder essentials

`__init__` `__repr__` `__str__` `__eq__` `__len__` `__add__`

Method types

instance → `self` · `@classmethod` → `cls` · `@staticmethod` → neither

Modern shortcut

`@dataclass` for data-holding classes

Summary notes

Object-oriented programming in Python organizes code around **classes** (blueprints) and **objects** (instances). The four pillars — **encapsulation**, **inheritance**, **polymorphism**, and **abstraction** — give you tools to control access, share code, vary behavior, and define contracts. In practice, Pythonic OOP leans on dunder methods for native-feeling objects, properties for validated access, class/static methods for construction and utilities, and composition over deep inheritance whenever the relationship isn't a true "is-a." Multiple inheritance is powerful but should be used deliberately, understanding the MRO; mixins and dataclasses reduce boilerplate; and design patterns are tools to reach for only when they genuinely simplify your design.

Continue learning

BOOKS

- "Fluent Python" — Luciano Ramalho
- "Python Object-Oriented Programming" — Dabeaz-adjacent, Steven Lott
- "Architecture Patterns with Python" — Percival & Gregory

DOCUMENTATION

- Python Docs: Classes (docs.python.org/3/tutorial/classes.html)
- Python Docs: `dataclasses` module
- Python Docs: `abc` — Abstract Base Classes

RESEARCH & DEEP DIVES

- PEP 3119 — Introducing Abstract Base Classes
- PEP 557 — Data Classes

COMMUNITIES

- r/learnpython
- Python Discord
- Stack Overflow — `[python]` `[oop]` tags

- "The Art of the Metaobject Protocol" (advanced OOP theory)

PRACTICE PLATFORMS

- Exercism — Python track
- LeetCode — Object-Oriented Design problems
- Codewars — OOP kata

SEARCH KEYWORDS

python dunder methods

MRO C3 linearization

composition over inheritance

python dataclass vs class

AI PROMPTS FOR FURTHER LEARNING

- "Explain Python's MRO with a diamond inheritance example"
- "Refactor this class hierarchy to use composition instead"
- "Quiz me on Python dunder methods with code snippets"